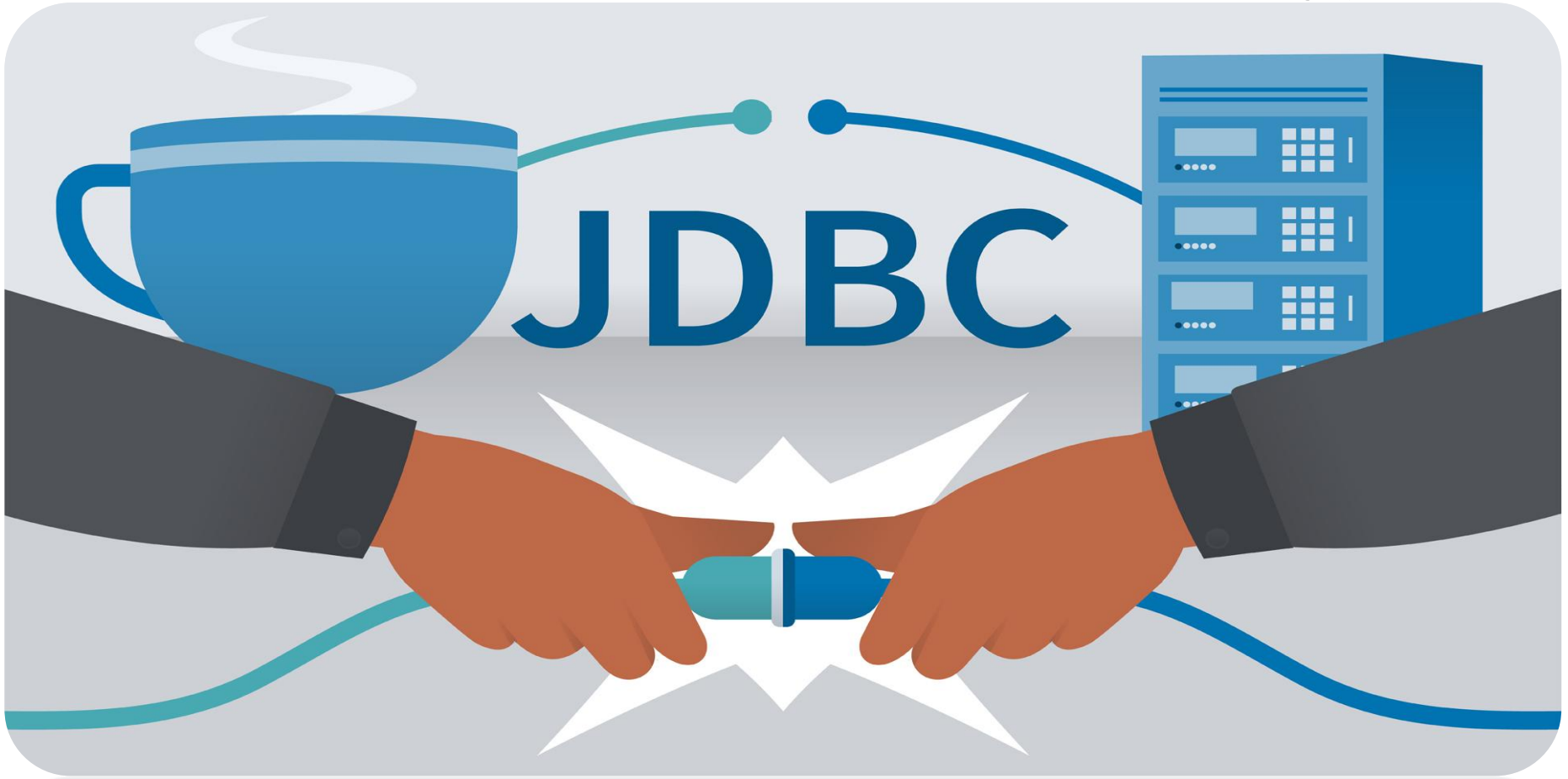
CHAPITRE 5

Manipulation de la Base de Données avec JDBC

Application JAVA < = > DB-MySQL !?

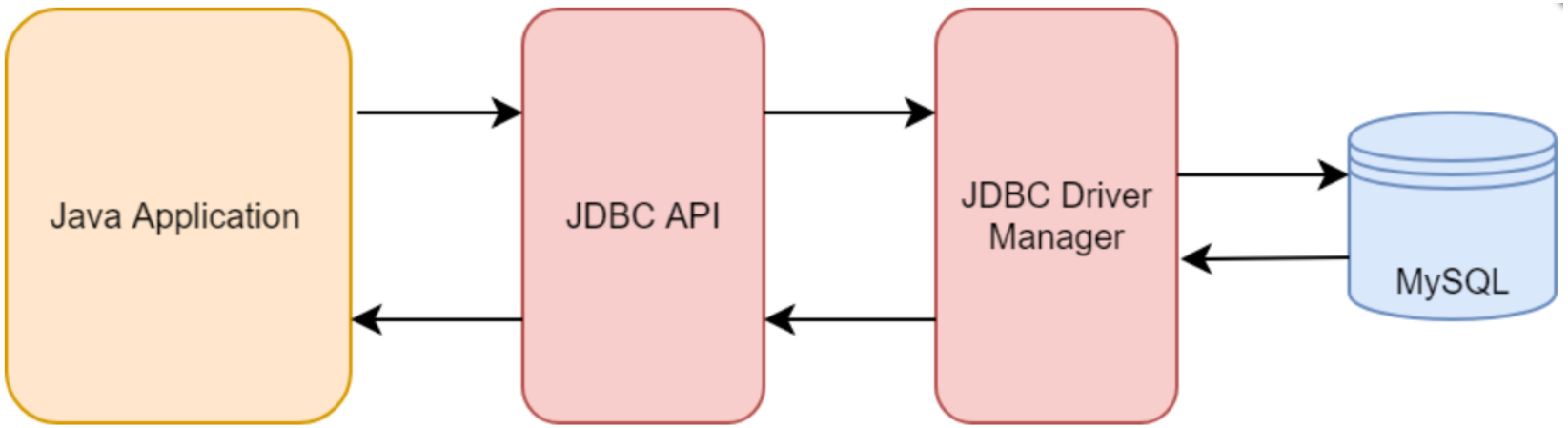
- ✓ Connecter une base de données en Java <=> **Java Data Base Connectivity** (**JDBC**)



What is JDBC ?



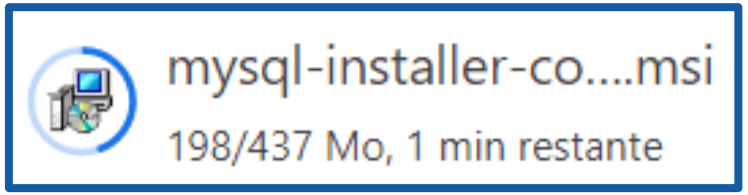
- ✓ JDBC : Java Database Connectivity.
- ✓ JDBC est une API fournie avec Java permettant de se connecter à des bases de données.
- ✓ JDBC est un moyen d'émettre des requêtes SQL vers les serveurs, et de récupérer le résultat de cette requête dans du code Java.



Install / Config JDBC

- ✓ Télécharger le driver JDBC Java mysql Connector :

  https://dev.mysql.com/downloads/connector/j/



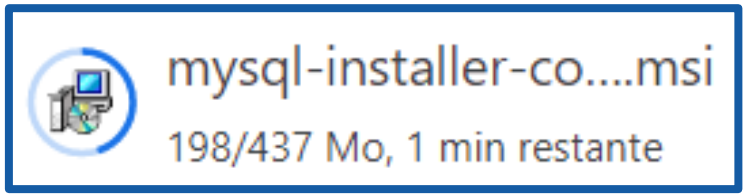
- ✓ Installer et configurer le driver **Java Mysql Connector**
- ✓ Créer le script nécessaire pour assurer une connexion Java avec MySQL



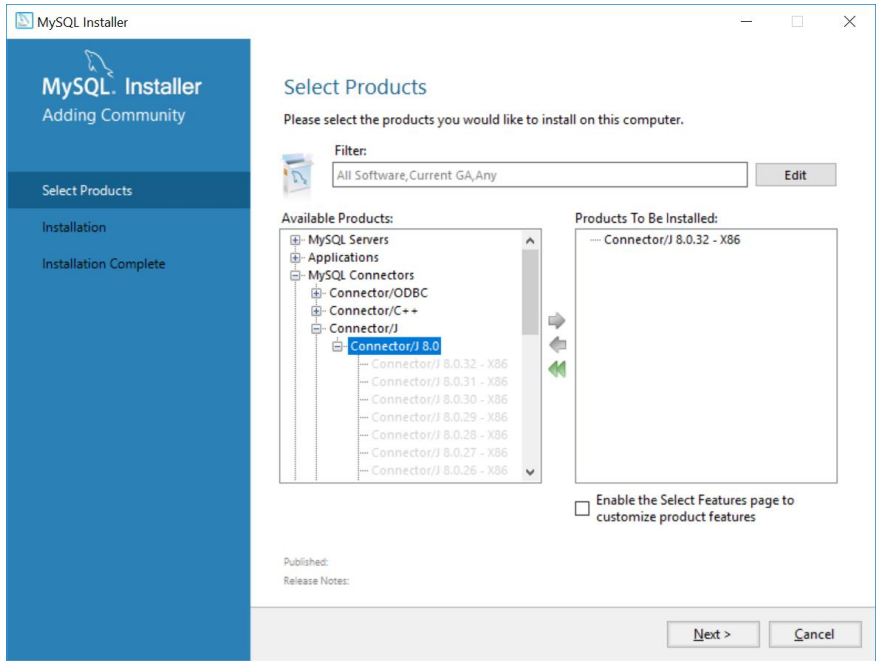
Install / Config JDB

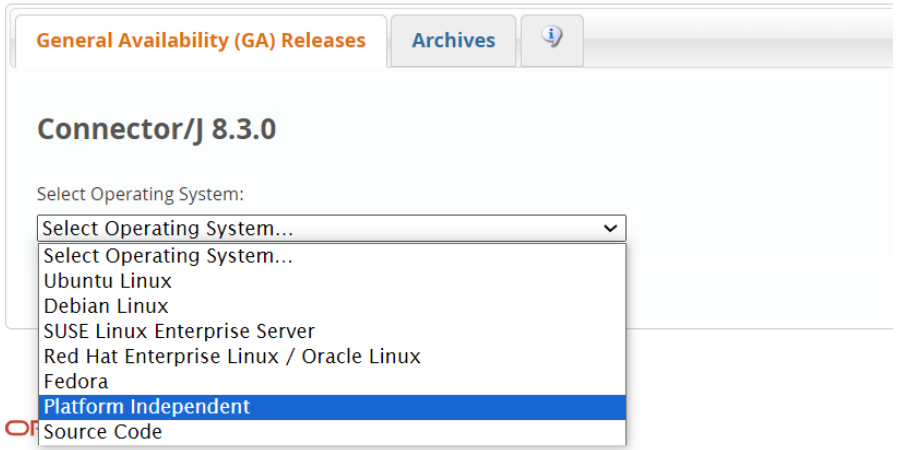
- ✓ Télécharger le driver JDBC Java mysql Connector :

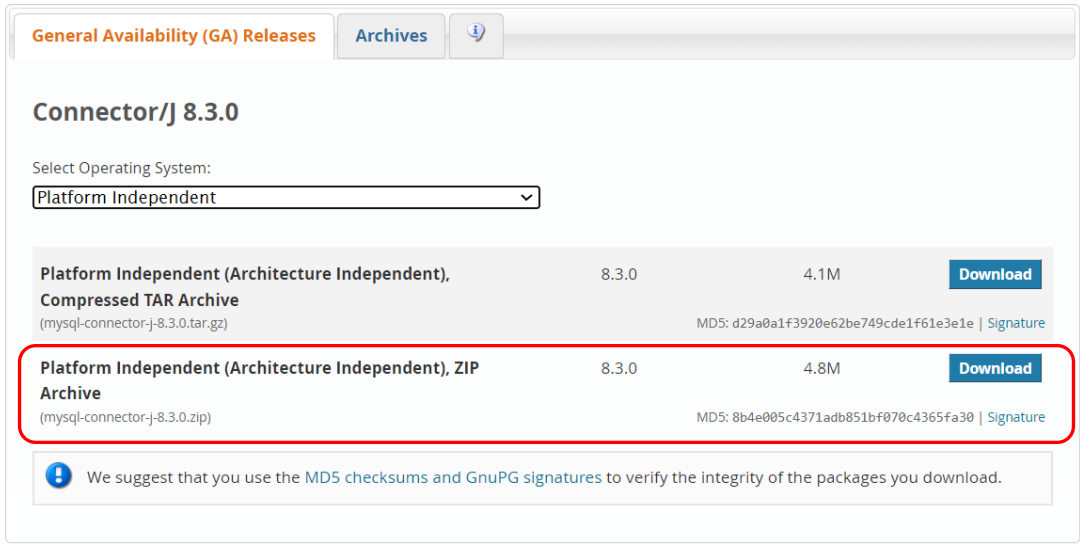
https://dev.mysql.com/downloads/connector/j/



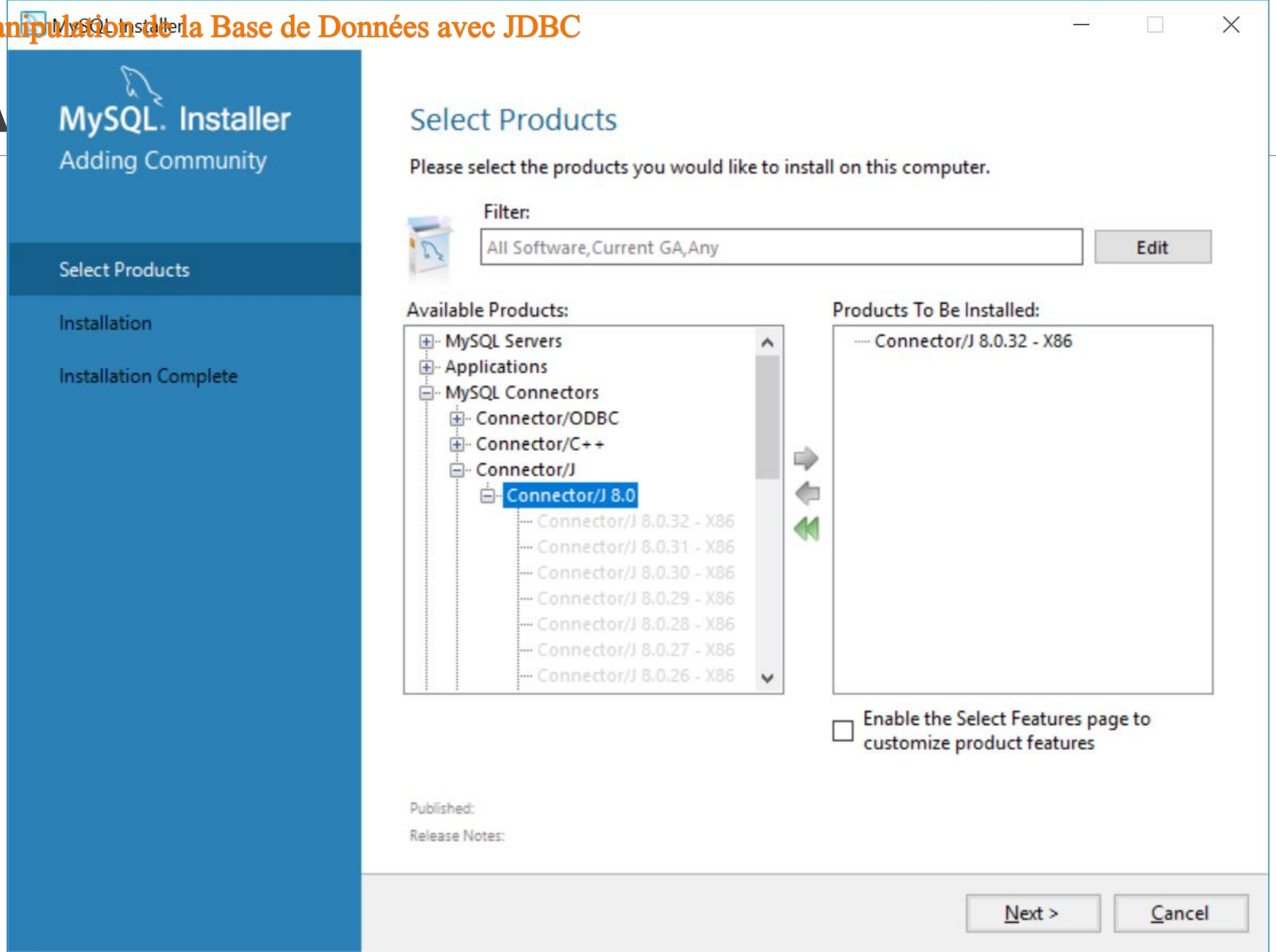
- ✓ Installer et configurer le driver **Java Mysql Connector**



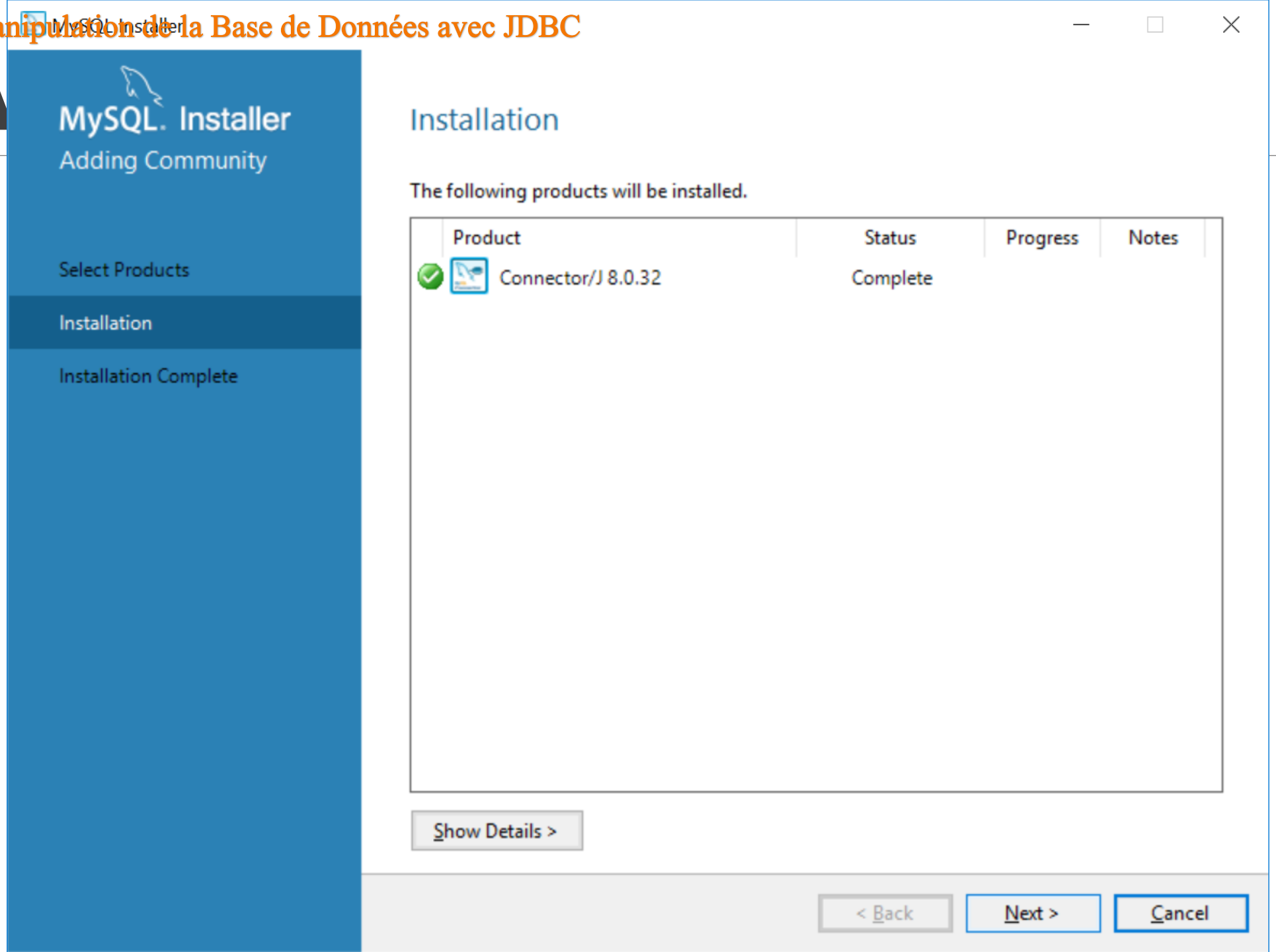




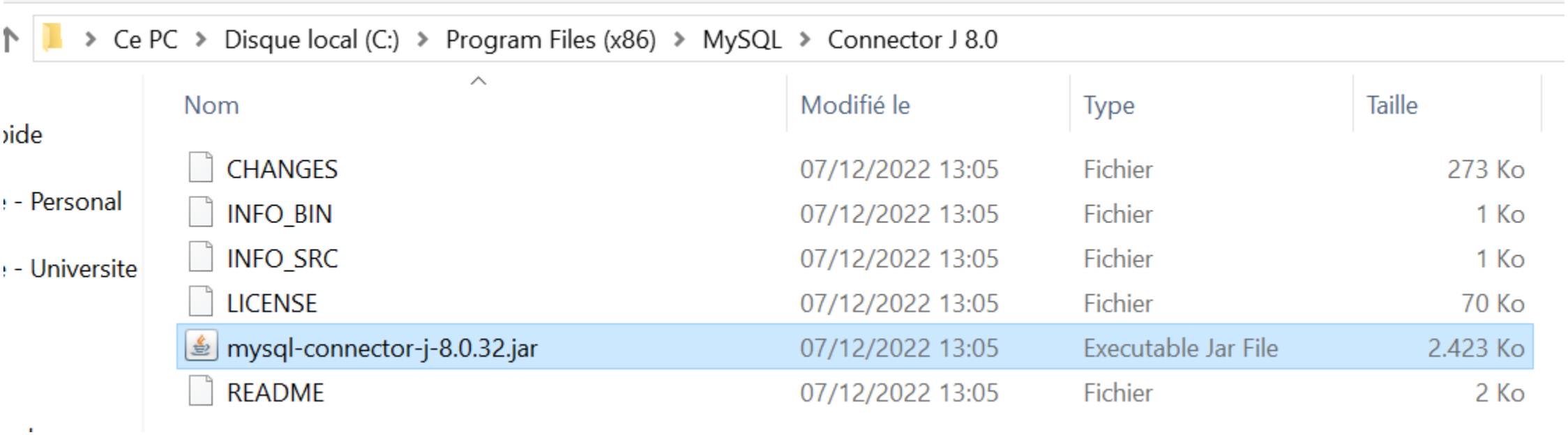
A


MySQL Installer
MySQL. Installer
Adding Community
Select Products
Installation
Installation Complete
Select Products
Please select the products you would like to install on this computer.
Filter:
All Software,Current GA,Any
Edit
Available Products:
MySQL Servers
Applications
MySQL Connectors
Connector/ODBC
Connector/C++
Connector/J
Connector/J 8.0
Connector/J 8.0.32 - X86
Connector/J 8.0.31 - X86
Connector/J 8.0.30 - X86
Connector/J 8.0.29 - X86
Connector/J 8.0.28 - X86
Connector/J 8.0.27 - X86
Connector/J 8.0.26 - X86
Products To Be Installed:
Connector/J 8.0.32 - X86
Enable the Select Features page to customize product features
Published:
Release Notes:
Next >
Cancel

A

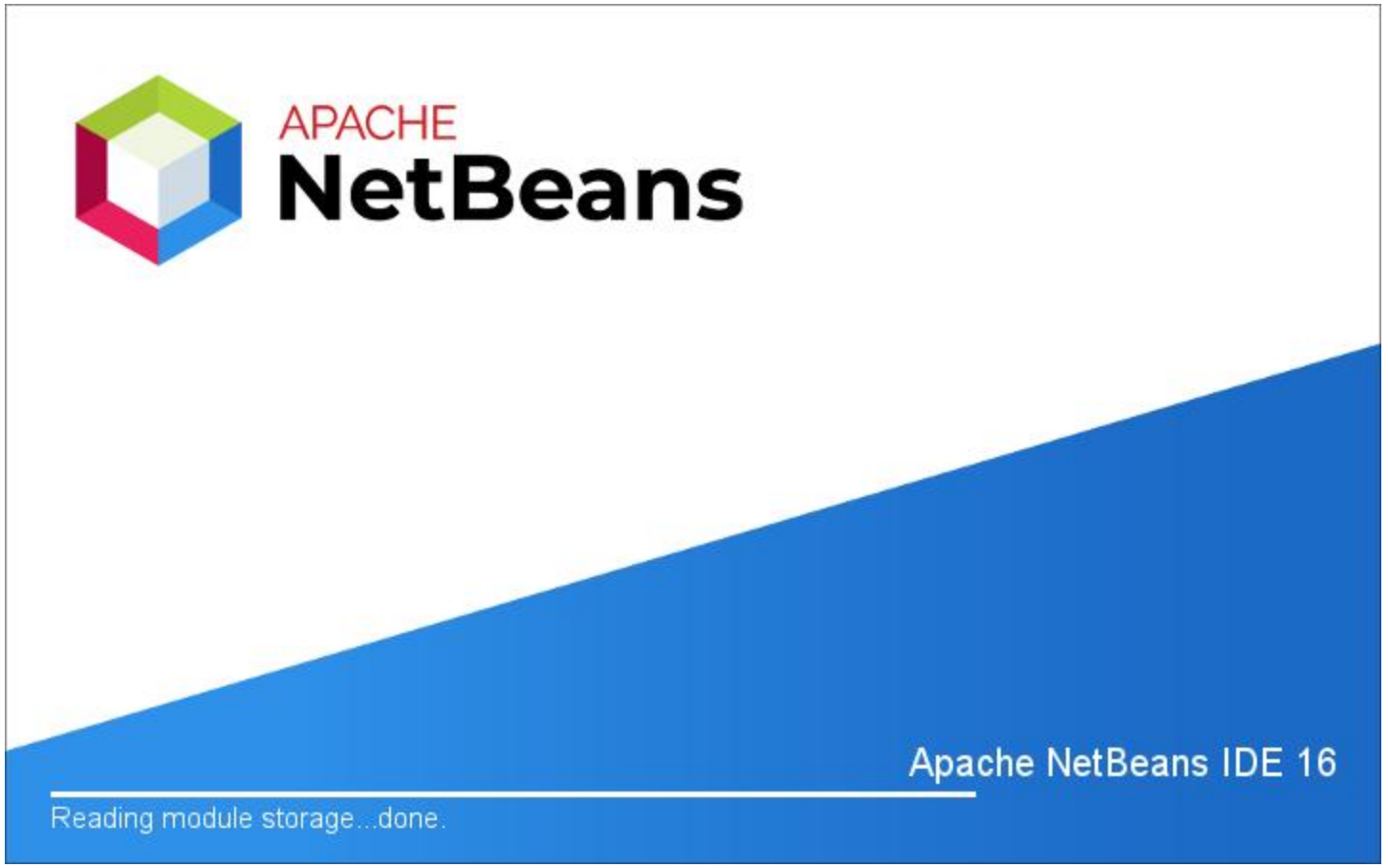

MySQL Installer
MySQL. Installer
Adding Community
Select Products
Installation
Installation Complete
Installation
The following products will be installed.
Product
Status
Progress
Notes
Connector/J 8.0.32
Complete
Show Details >
< Back
Next >
Cancel

Accès avec JDBC

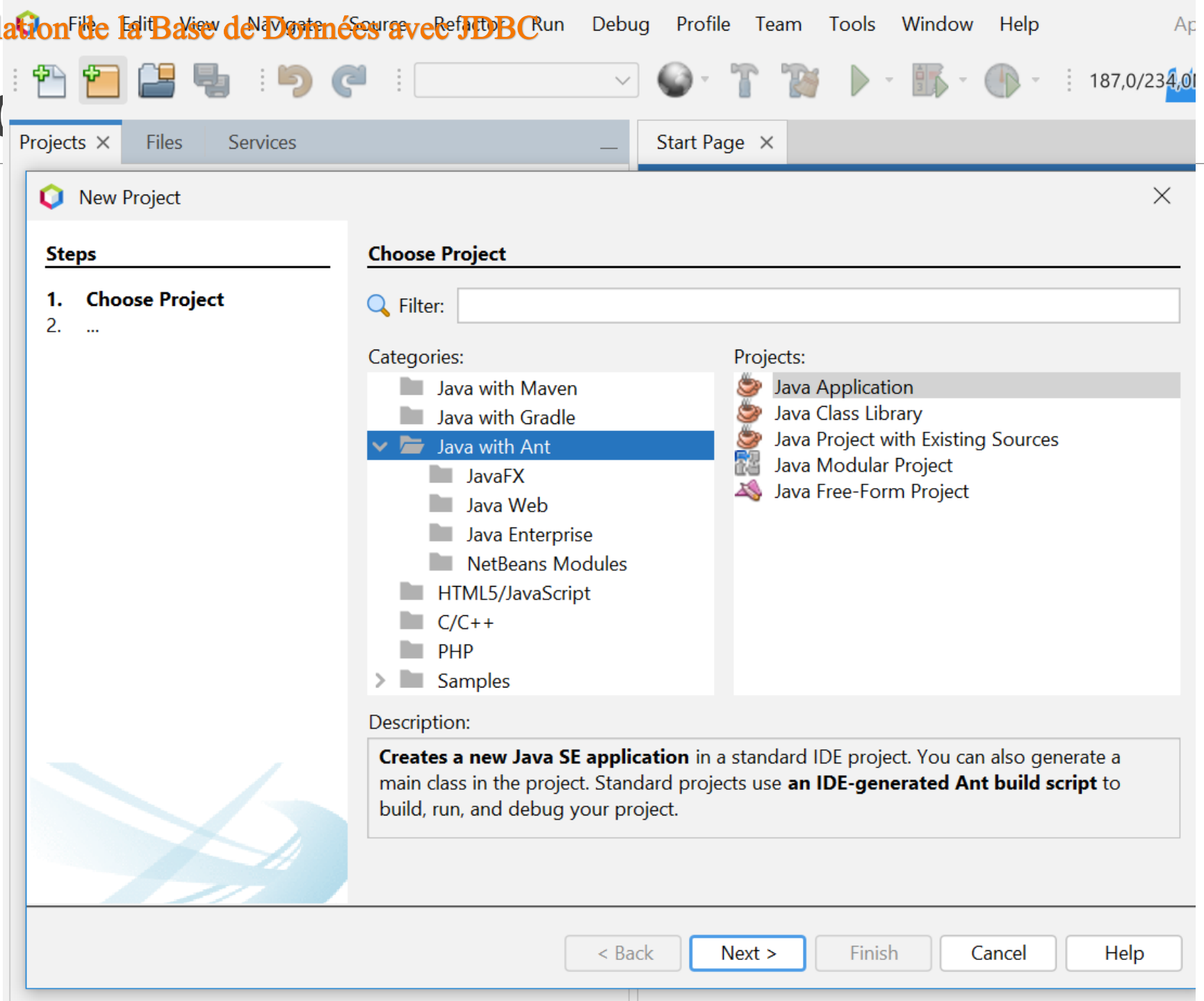


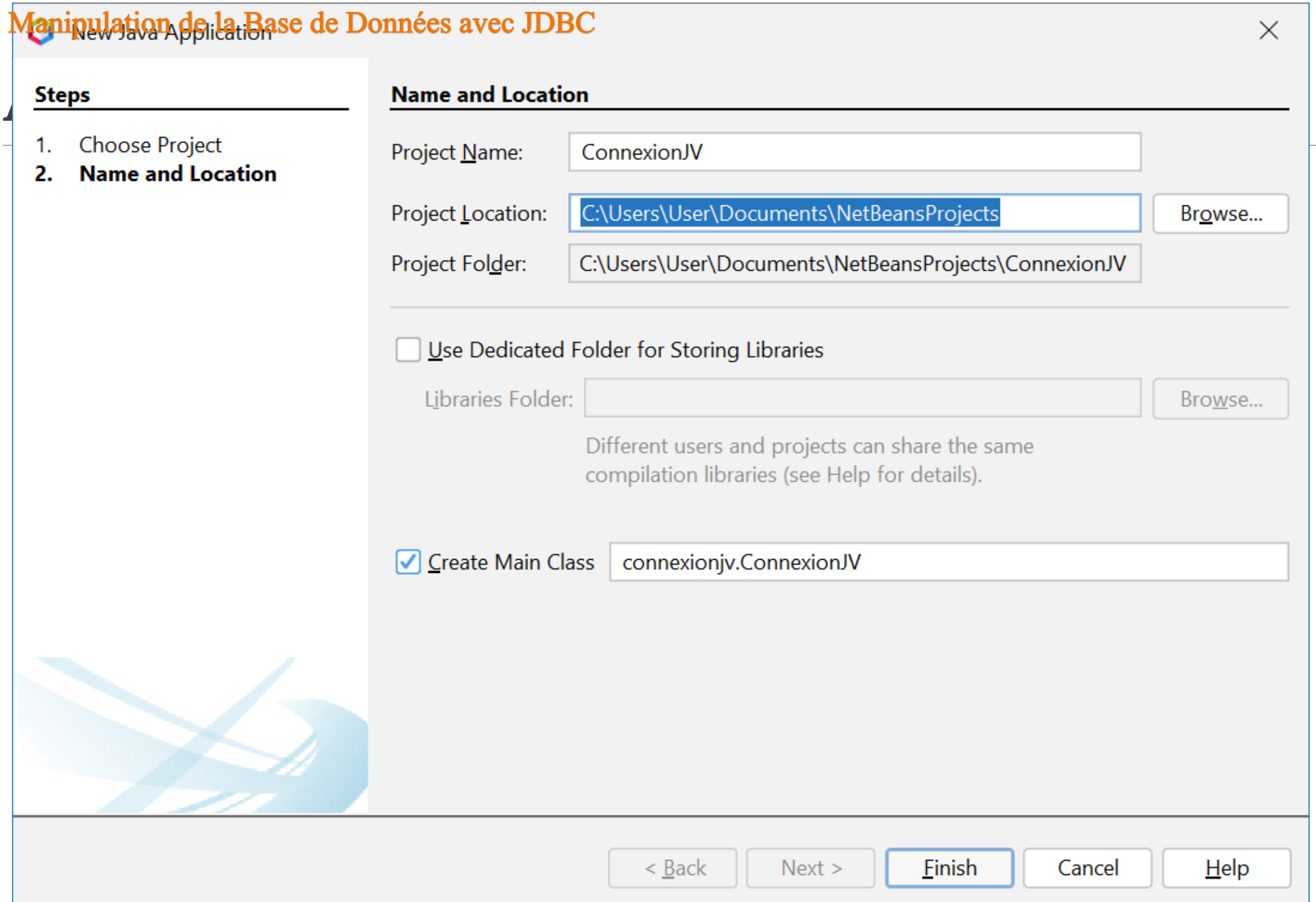
- ✓ Le fichier **mysql-connector-j-8.0.32.jar** est le fichier ciblé à l'intégrer dans les projets **JavaApp**
- ✓ Créer le script nécessaire pour assurer une connexion Java avec MySQL.

Accès avec JDBC

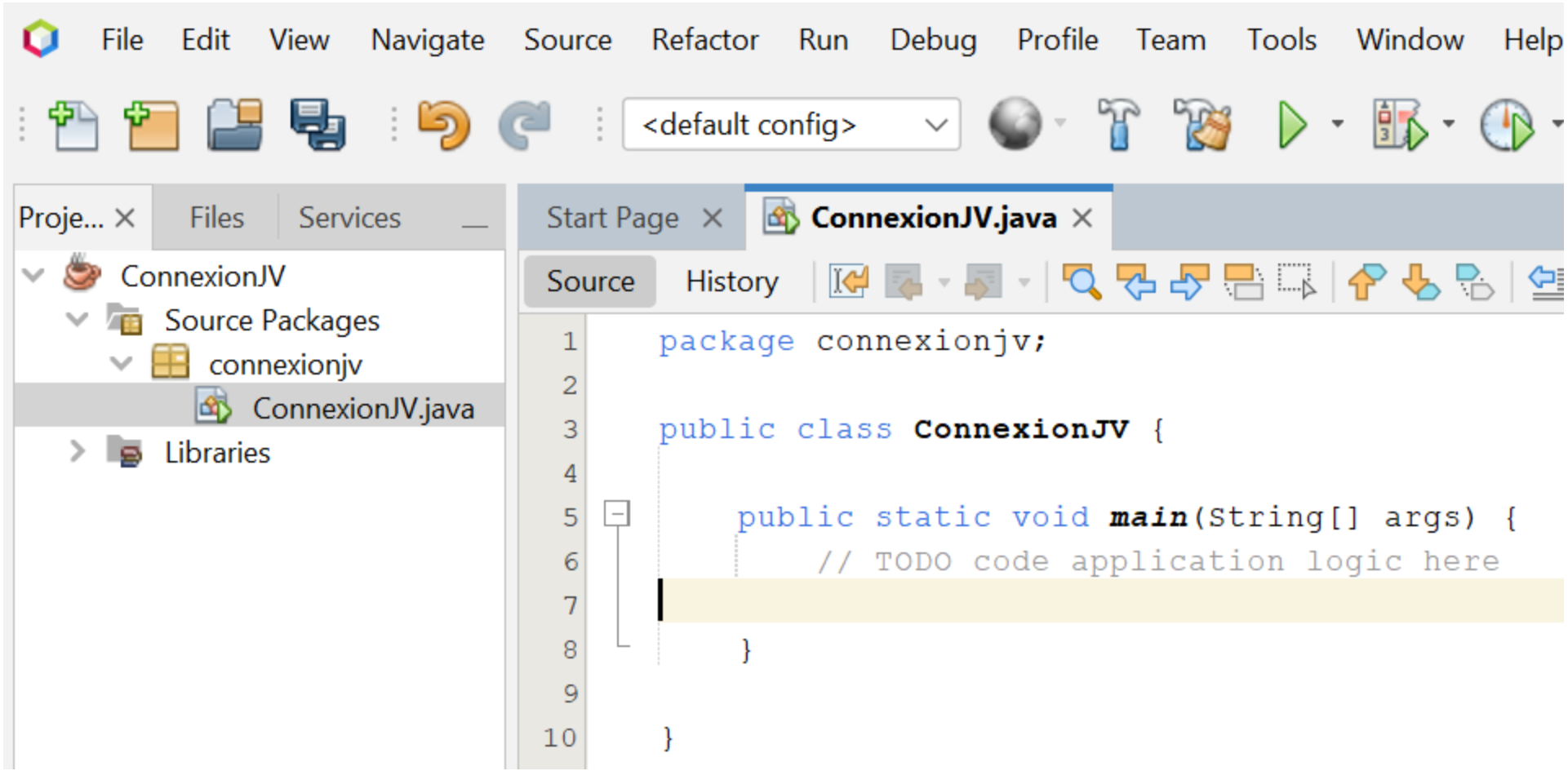


Acc


File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
Projects
Files
Services
Start Page
New Project
Steps
1. Choose Project
2. ...
Choose Project
Filter:
Categories:
Java with Maven
Java with Gradle
Java with Ant
JavaFX
Java Web
Java Enterprise
NetBeans Modules
HTML5/JavaScript
C/C++
PHP
Samples
Projects:
Java Application
Java Class Library
Java Project with Existing Sources
Java Modular Project
Java Free-Form Project
Description:
Creates a new Java SE application in a standard IDE project. You can also generate a main class in the project. Standard projects use an IDE-generated Ant build script to build, run, and debug your project.
< Back
Next >
Finish
Cancel
Help

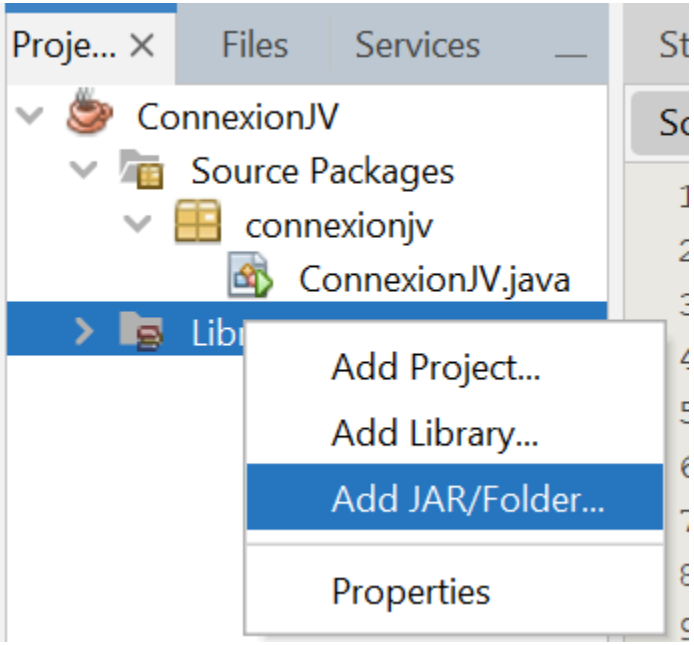

New Java Application
Steps
1. Choose Project
2. Name and Location
Name and Location
Project Name: ConnexionJV
Project Location: C:\Users\User\Documents\NetBeansProjects
Browse...
Project Folder: C:\Users\User\Documents\NetBeansProjects\ConnexionJV
Use Dedicated Folder for Storing Libraries
Libraries Folder:
Browse...
Different users and projects can share the same compilation libraries (see Help for details).
Create Main Class connexionjv.ConnexionJV
< Back
Next >
Finish
Cancel
Help

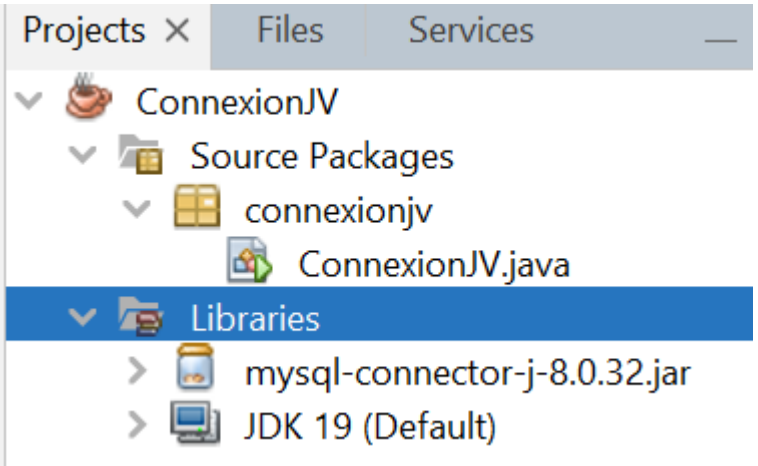
Accès avec JDBC

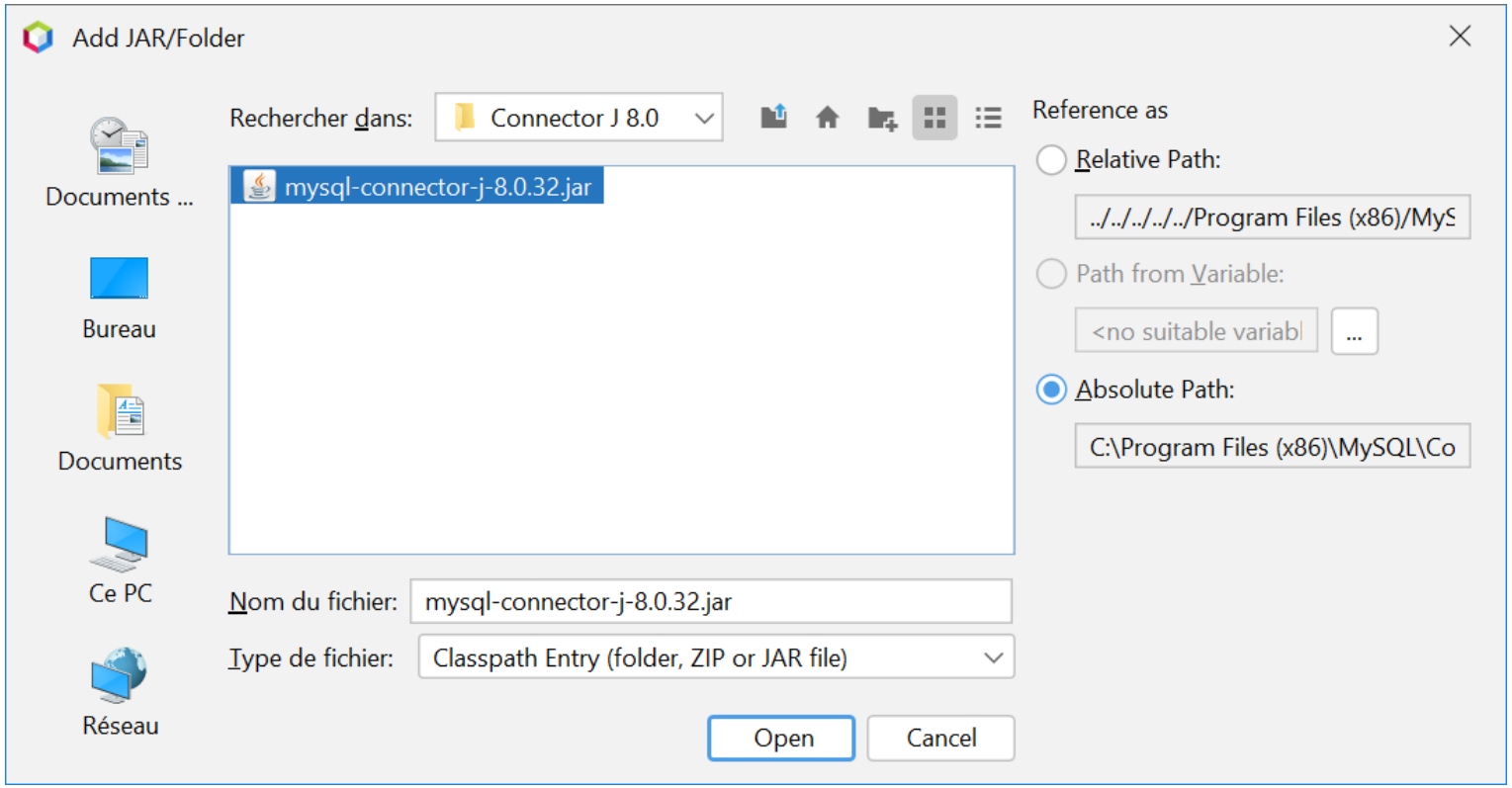


Accès avec JDBC

Il faut ajouter le fichier driver dans le projet crée



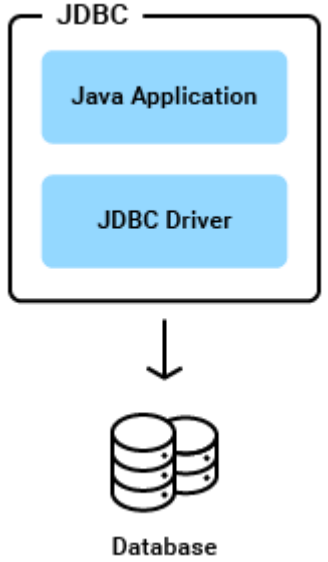




Accès avec JDBC

Le processus d'utilisation du JDBC se compose de quatre actions :

- ✓ Importer le package **java.sql.***
- ✓ **Chargement** du **pilote** de la base.
- ✓ Ouverture d'une **connexion** à la base de données. Le code doit fournir au minimum trois informations :
  i. une URL d'accès,
  ii. un nom de connexion
  iii. un mot de passe.
- ✓ Création d'un objet de type **Statement**, qui va prendre en charge l'acheminement de la requête SQL vers le serveur, et récupérer les résultats.
- ✓ Récupération du résultat de la requête dans un objet de type **ResultSet**, et exploitation de ce résultat.



5 Steps to connect AppJava <=> DB

Il existe 5 étapes pour connecter une application Java à une base de données à l'aide de JDBC. Ces étapes sont les suivantes:

A. Enregistrer la classe de pilote
B. Créer une connexion
C. Créer un objet *Statement*
D. Exécuter des requêtes
E. Fermer la connexion

5 Steps : *A. Enregistrer la classe de pilote*

- La méthode ***forName()*** de la classe '*Class*' est utilisée pour enregistrer le pilote. Cette méthode est utilisée pour charger dynamiquement la classe de pilote.

```
Class.forName("DB.jdbc.driver.DBDriver");
```

- *Exemple:*

Un programme Java charge le pilote *MySQL* pour établir une connexion à la base de données :

```
// Chargement de la classe par son nom
Class c = Class.forName( className:"com.mysql.jdbc.Driver") ;
```

5 Steps : *B. Créer une connexion*

- La méthode ***getConnection()*** de la classe '*DriverManager*' est utilisée pour établir la connexion avec la base de données.

```
// Connexion
Connection con = DriverManager.getConnection(
    url:conString, user:nomConnexion, password:motDePasse) ;
```

- Exemple :

```
String protocole =  "jdbc:mysql:" ;
// Adresse IP de l'hôte de la base et port
String ip =  "localhost" ;  // dépend du contexte
String port =  "3306" ;  // port MySQL par défaut
// Nom de la base ;
String nomBase =  "DBTest" ;  // dépend du contexte
// Chaîne de connexion
String conString = protocole +  "//" + ip +  ":" + port +  "/" + nomBase ;
// Identifiants de connexion et mot de passe
String nomConnexion =  "root" ;  // dépend du contexte
String motDePasse =  "Pass.Wrd" ;  // dépend du contexte
```

```
// Connexion
Connection con = DriverManager.getConnection(
    url:conString, user:nomConnexion, password:motDePasse) ;
```

5 Steps : *C. Créer un objet Statement*

- La méthode ***createStatement()*** de l'interface '*Connection*' est utilisée pour créer un objet *Statement*.
- L'objet ***Statement*** est responsable de l'exécution des requêtes avec la base de données.

Statement *StmtName* = ConName.**createStatement()**;

- *Exemple:*

La création d'un objet *Statement* :

```
Statement smt = con.createStatement() ;
```

5 Steps : *D. Exécuter des requêtes*

- La méthode ***executeQuery()*** de l'interface '*Statement*' est utilisée pour exécuter des requêtes dans une base de données.
- Cette méthode renvoie l'objet de ***ResultSet*** qui peut être utilisé pour obtenir tous les enregistrements d'une table.

```
ResultSet res = StmtName.executeQuery("SQL Request");
```

- *Exemple :*

Créer un objet *ResultSet* pour récupérer la liste des étudiants enregistrés dans la base de données :

```
// Envoi d'un requête de séléction
String sql =  "select * from etudiant" ;

ResultSet rs = smt.executeQuery( string:sql) ;

while (rs.next()) {
    System.out.println( x: rs.getString( string:"nom")) ;
}
```

5 Steps : *E. Fermer la connexion*

- ✓ En fermant la connexion, *ResultSet* sera fermé automatiquement.
- ✓ La méthode ***close()*** de l'interface '*Connection*' est utilisée pour fermer la connexion.

```
ConnName.close( );
```

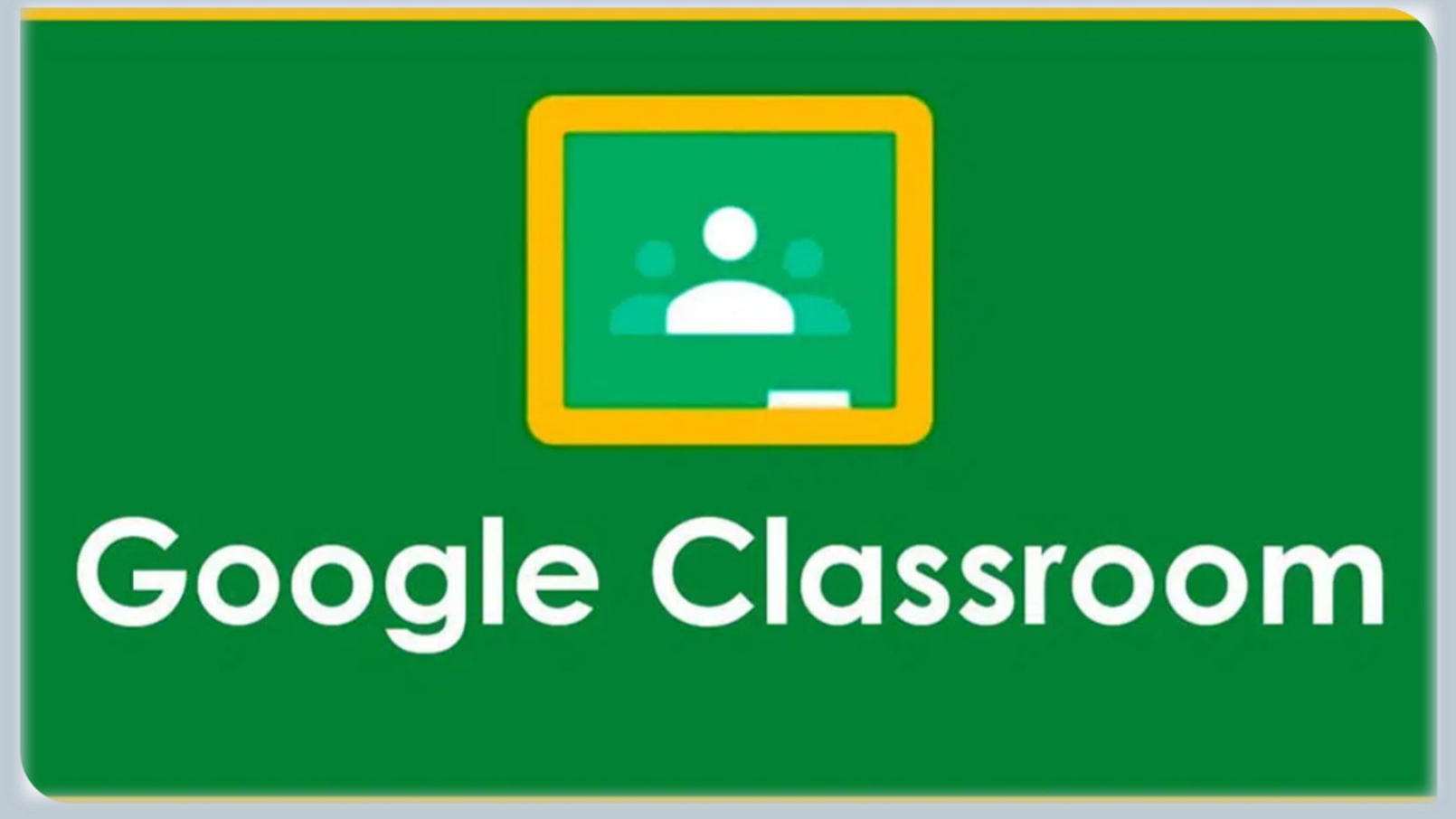
- ✓ *Exemple:*

```
con.close();
```

5 Steps to connect AppJava <=> DB

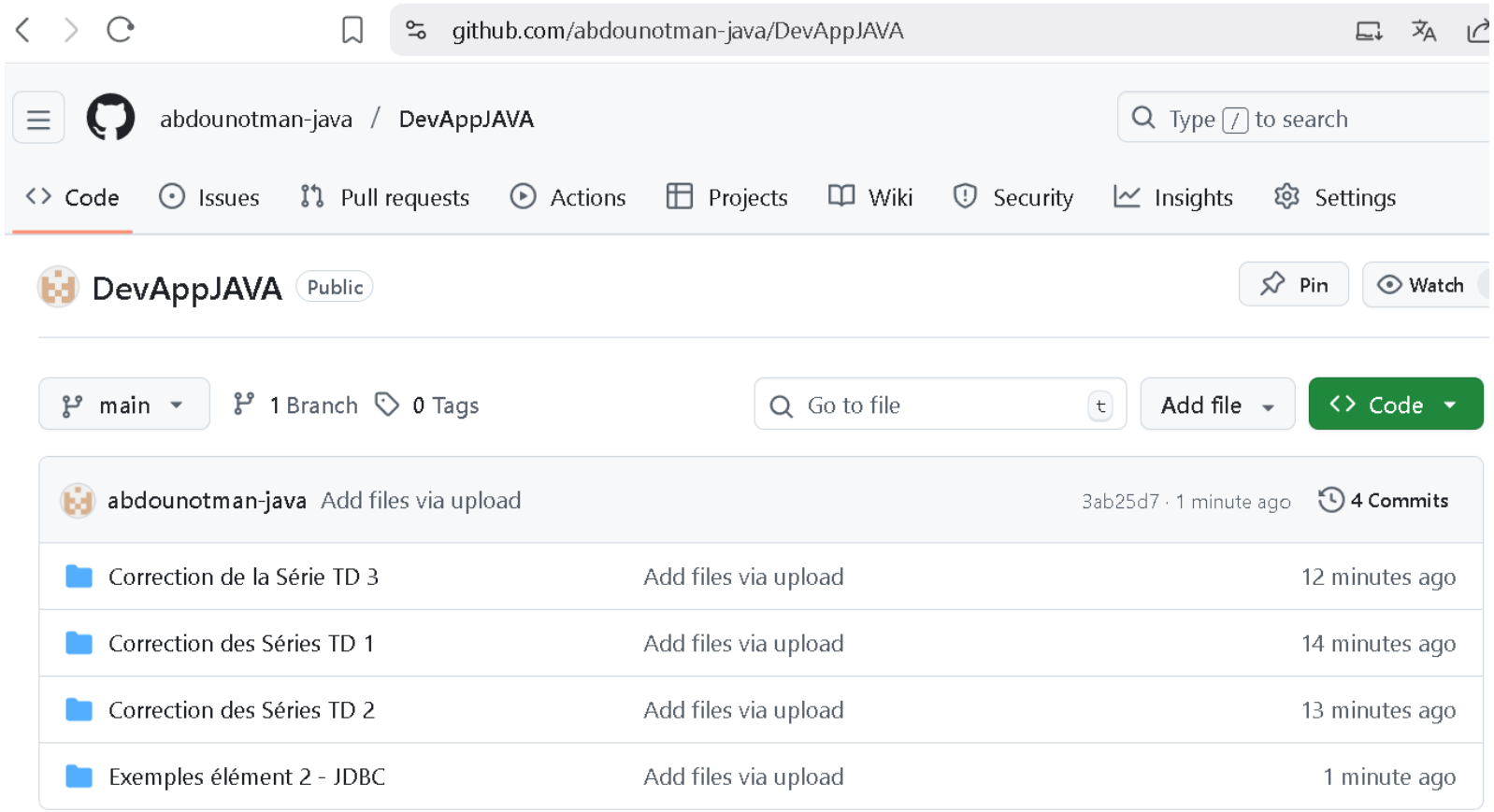
```
// A. Enregistrer la classe de pilote
Class c = Class.forName( className:"com.mysql.jdbc.Driver") ;
 // Protocole de connexion
String protocole =  "jdbc:mysql:" ;
 // Adresse IP de l'hôte de la base et port
String ip =  "localhost" ;  // dépend du contexte
String port =  "3306" ;  // port MySQL par défaut
 // Nom de la base ;
String nomBase =  "DBTest" ;  // dépend du contexte
 // Chaîne de connexion
String conString = protocole +  "//" + ip +  ":" + port +  "/" + nomBase ;
 // Identifiants de connexion et mot de passe
String nomConnexion =  "root" ;  // dépend du contexte
String motDePasse =  "FS.Tetouan_2023" ;  // dépend du contexte
 // B.  Créer une connexion
Connection conct = DriverManager.getConnection(
    url:conString,  user:nomConnexion,  password:motDePasse) ;
//  C. Créer un objet Statement
Statement smt = conct.createStatement() ;
 // D. Exécuter des requêtes
String sql =  "select * from etudiant" ;
ResultSet rs = smt.executeQuery( string:sql) ;
 while (rs.next())
   System.out.println( x:rs.getString( string:"nom")) ;
conct.close();
```

```
Output - ConnexionJV (run)
run:
Alami Samir
Mssiri Karima
BUILD SUCCESSFUL (total time: 0 seconds)
```



Pour toutes informations ou supports pédagogiques, veuillez consulter le **Google Classroom** dédié :

https://classroom.google.com/u/2/c/ODg1ODU3OTk2Njg5



All ressources in GITHUB

HTTPS://GITHUB.COM/ABDOUNOTMAN-JAVA/DEVAPPJAVA